Internetrecherche

Aufgabenblatt 2

Modul: T4INF2003 – Software Engineering I
Kurs: TINF2024
Dozentin: Charlotte Chichlow
Semester: Frühjahr 2026

Projekt: Entwicklung eines Online Aufgaben-Planungssystems

1. Einleitung 2
2. Allgemeines 2
   2.1 Einsatzumfeld 2
   2.2 Grundlogik einer prototypischen Applikation 3
   2.3 Terminologie (Glossar-Vorarbeit) 3
   2.4 Problemstellung und Systemabgrenzung 3
3. Übersicht über themenrelevante Applikationen 4
   3.1 TimeHero 4
   3.2 Motion 4
   3.3 Timeflow 5
   3.4 LiquidPlanner / Tempo Portfolio Manager 5
4. IT-Spezifika themenrelevanter Applikationen 7
   4.1 Allgemeine technologische Komponenten 7
   4.2 Technologien im eigenen Projekt & Team-Kompetenz (Projekt TeaPot) 8
   4.3 Sicherheit, Datenschutz und Non-Functional Requirements 8
5. Analyse einer konkreten Applikation: Motion 8
   5.1 Leistungsmerkmale 9
   5.2 Use-Cases (Anwendungsfälle) 9
6. UML-Diagramme 10
7. Fazit 11
Literatur- und Quellenverzeichnis 12

1. Einleitung

Im Rahmen der Veranstaltung „Software Engineering I“ wird ein praxisnahes Projekt durchgeführt, in dem ein Software-System von der Anforderungsanalyse über die Modellierung bis hin zur prototypischen Implementierung entwickelt werden soll. Ziel des vorliegenden Projekts der Gruppe „TeaPot“ ist die Entwicklung eines intelligenten Online Aufgaben-Planungssystems. Dieses System soll Organisationen, Teams und Einzelpersonen dabei unterstützen, anfallende Aufgaben nicht nur zentral zu erfassen, sondern auch effizient und automatisiert zeitlich zu planen.

Im Gegensatz zu klassischen, statischen To-Do-Listen sollen bei diesem System Parameter wie Prioritäten, harte Deadlines sowie die individuell verfügbare Arbeitszeit der jeweiligen Benutzer berücksichtigt werden. Auf Basis dieser Daten wird ein dynamischer und sinnvoller Arbeitsplan generiert. Das System wird als browserbasierte Plattform konzipiert. Die Plattform ist primär auf die individuelle, ressourcenbasierte Optimierung von Zeitplänen für Teams und Einzelpersonen ausgerichtet.

Ziel dieses Recherchenberichts ist es, im Vorfeld der eigentlichen Entwicklungsarbeit eine fundierte Marktanalyse durchzuführen. Hierbei wird ein Überblick über bestehende, ähnliche Lösungen auf dem Markt gegeben. Eine ausgewählte Konkurrenzanwendung wird detaillierter analysiert, um Best Practices zu identifizieren. Zudem werden die zentralen Begriffe (als Vorarbeit für das Glossar) und die technischen Grundlagen des Projekts definiert. Die hier gewonnenen Erkenntnisse bilden das Fundament für die anstehende Erstellung des Lasten- und Pflichtenhefts.

2. Allgemeines

2.1 Einsatzumfeld

Online Aufgaben-Planungssysteme finden in nahezu allen Bereichen Anwendung, in denen arbeitsteilige Prozesse stattfinden und mehrere Personen koordiniert an gemeinsamen Zielen arbeiten. Typische Einsatzbereiche umfassen klassische Unternehmensstrukturen, agile Projektteams, aber auch ehrenamtliche Vereine, studentische Arbeitsgruppen sowie Start-ups.

In diesen kollaborativen Umgebungen müssen täglich Aufgaben delegiert und priorisiert werden. Dabei kommt es häufig zu Ressourcenengpässen, da Faktoren wie die Wichtigkeit einer Aufgabe (Priorität), der späteste Abgabetermin (Deadline), Abhängigkeiten zwischen verschiedenen Arbeitspaketen und die real verfügbare Zeit der Mitarbeiter in Einklang gebracht werden müssen. Während viele Organisationen bereits rudimentäre digitale Tools nutzen, stoßen diese oft an ihre Grenzen, wenn es um die dynamische Umplanung geht. Moderne Systeme adressieren dieses Problem durch Algorithmen, die Aufgaben automatisiert in verfügbare Zeiträume einplanen und bei Verzögerungen den Plan selbstständig anpassen. Das Projekt TeaPot zielt exakt auf diese Marktlücke der intelligenten und ressourcenbasierten Aufgabenplanung ab.

2.2 Grundlogik einer prototypischen Applikation

Eine prototypische Applikation in diesem Themenumfeld folgt in der Regel einem festen logischen Ablauf, der auch für das geplante System adaptiert wird:

Zunächst können sich Einzelpersonen oder Teams registrieren. Daraufhin können sich einzelne Benutzer anmelden, die durch einen Administrator der jeweiligen Team-Gruppe zugeordnet werden und entsprechende Rollen und Rechte erhalten.

Im operativen Betrieb werden Aufgaben (Tasks) im System angelegt. Diese sind nicht nur einfache Textfelder, sondern komplexe Datenobjekte, die mit Eigenschaften wie Priorität, geschätztem Zeitaufwand und einer verbindlichen Deadline versehen werden. Diese Aufgaben können gezielt an Mitglieder delegiert werden. Ein zentraler Baustein der Logik ist das individuelle „Arbeitsprofil“ der Benutzer: Hier hinterlegen die Anwender ihre regulären Arbeitszeiten, geplante Pausen sowie maximale Arbeitsintervalle pro Sitzung.

Das System nutzt eine Planungslogik, um diese Metadaten der Aufgaben mit den Arbeitsprofilen abzugleichen. Das Resultat ist ein automatisch generierter Zeitplan, der den Benutzern in einer übersichtlichen Kalender- oder Dashboard-Ansicht präsentiert wird.

2.3 Terminologie (Glossar-Vorarbeit)

Für das einheitliche Verständnis innerhalb der Projektgruppe und in der Kommunikation mit den Stakeholdern werden folgende zentrale Begriffe definiert:

- **Team/Projekt-Gruppe:** Eine logische Einheit innerhalb der Plattform, deren Mitglieder gemeinsam Aufgaben bearbeiten und deren Verfügbarkeiten berücksichtigt werden (z. B. ein Scrum-Team oder eine studentische Arbeitsgruppe).
- **Benutzer (User):** Eine registrierte Person, die sich authentifizieren kann und Aufgaben innerhalb ihrer zugewiesenen Organisation bearbeitet.
- **Administrator:** Ein Benutzer mit erweiterten systemseitigen Privilegien, der berechtigt ist, Team-Einstellungen vorzunehmen, neue Mitglieder einzuladen und systemweite Parameter zu verwalten.
- **Aufgabe (Task):** Die atomare Arbeitseinheit des Systems. Sie beinhaltet mindestens einen Titel, eine detaillierte Beschreibung sowie planungsrelevante Metadaten.
- **Priorität:** Ein quantifizierbarer oder kategorischer Wert (z. B. Niedrig, Mittel, Hoch), der dem Planungsalgorithmus signalisiert, wie wichtig die zeitnahe Erledigung im Konfliktfall ist.
- **Deadline:** Ein fest definierter Zeitpunkt (Datum und Uhrzeit), bis zu dem eine Aufgabe zwingend abgeschlossen sein muss.
- **Arbeitsprofil:** Ein Datensatz, der die individuellen Kapazitäten eines Benutzers beschreibt (z. B. Kernarbeitszeiten, Abwesenheiten, Pausenregelungen).
- **Arbeitsplan / Auto-Schedule:** Die vom System berechnete, zeitliche Verteilung von Aufgaben auf die verfügbaren Zeitslots der Benutzer.

2.4 Problemstellung und Systemabgrenzung

Klassische Kalenderanwendungen (wie Google Calendar oder Microsoft Outlook) und einfache To-Do-Listen (wie Microsoft To Do oder Trello) stoßen im professionellen Projektumfeld schnell an ihre Grenzen. Das Kernproblem besteht darin, dass diese beiden Welten meist getrennt voneinander existieren: Eine Aufgabe in einer To-Do-Liste hat kein konkretes Zeitfenster, und ein Termin im Kalender spiegelt keinen Aufgabenfortschritt wider.

Sobald unvorhergesehene Ereignisse eintreten – etwa eine Krankmeldung, ein eskaliertes Kundenproblem oder ein ungeplant verlängertes Meeting – muss der Projektverantwortliche alle nachfolgenden Aufgaben manuell neu priorisieren und verschieben. Dieser ständige manuelle Planungsaufwand kostet im Arbeitsalltag enorm viel Zeit, erhöht die kognitive Belastung (Cognitive Load) und führt oft zu verpassten Deadlines. Das geplante System "TeaPot" grenzt sich daher bewusst von reinen Listen ab: Es übernimmt die kognitive Last der Zeitplanung und fungiert als intelligente Brücke zwischen Aufgabenmanagement und aktivem Kalendermanagement, indem es auf Basis von Algorithmen reaktiv auf Veränderungen antwortet und den Plan selbstständig optimiert.

Zur klaren Abgrenzung: TeaPot ist kein Ersatz für ein vollumfängliches ERP-System oder eine HR-Software zur Zeiterfassung für die Lohnabrechnung. Ebenso ist keine Integration von Videokommunikations-Tools geplant. Der Fokus liegt rein auf der algorithmischen Optimierung der individuellen und teambasierten Aufgabenplanung.

3. Übersicht über themenrelevante Applikationen

Um den Markt und bestehende Lösungsansätze zu verstehen, wurden vier etablierte Systeme analysiert, die ähnliche Problemstellungen adressieren.

3.1 TimeHero

TimeHero ist ein fortschrittliches Aufgaben- und Projektplanungssystem, das sich an Teams richtet, die viele termingebundene Aufgaben parallel bewältigen müssen. Das Alleinstellungsmerkmal von TimeHero ist die proaktive Einplanung: Das System analysiert bestehende Termine (z.B. durch Integration von Google Calendar oder Outlook), gleicht diese mit den Aufgabendeadlines ab und füllt die Lücken im Kalender automatisch mit Aufgaben. Verschiebt ein Mitarbeiter eine Aufgabe oder wird krank, plant TimeHero den restlichen Tag oder die Woche automatisch neu (Auto-Adjustment). Die Architektur basiert auf einer **Cloud-Lösung**, die plattformübergreifend über den Browser nutzbar ist und typischerweise einem Single-Page-Application (SPA) Muster (siehe 4.1) folgt, um die notwendige Geschwindigkeit für das Echtzeit-"Auto-Adjustment" zu gewährleisten.

3.2 Motion

Motion versteht sich als KI-gestützte "SuperApp" für das Zeit- und Aufgabenmanagement. Die Zielgruppe umfasst vor allem Wissensarbeiter, Manager und agile Teams. Die Web- und Mobile-App nutzt Algorithmen, um eine strikte Priorisierung vorzunehmen. Nutzer müssen lediglich Aufgaben in das System einpflegen, Deadlines setzen und die geschätzte Dauer angeben. Motion fungiert daraufhin als intelligenter Assistent, der die Aufgaben sekündlich

neu berechnet und als feste Blöcke in den Terminkalender des Nutzers legt. Die Architektur von Motion ist als **Web- und Mobile-App** konzipiert und basiert auf einer **Client-Server-Architektur**. Die rechenintensive, KI-gestützte Planungslogik (Algorithmus) läuft dabei primär serverseitig in der Cloud.

3.3 Timeflow

Timeflow ist ein stark visuell geprägtes Tool, das Kalender und klassische To-Do-Listen vereint. Es richtet sich eher an Freelancer und kleinere Teams. Im Gegensatz zu hochkomplexen Projektmanagement-Tools fokussiert sich Timeflow auf das sogenannte "Timeboxing". Anwender ziehen ihre Aufgaben per Drag-and-Drop in den Kalender, wobei das System durch visuelle Indikatoren (wie farbige Heatmaps) anzeigt, wann die tägliche Arbeitsbelastung eines Nutzers ein ungesundes Maß (Überlastung) erreicht. Die Architektur ist als **leichtgewichtige, plattformübergreifende Anwendung**ausgelegt, die auf einer hervorragenden User Experience (UX) basiert und typischerweise moderne Frontend-Technologien nutzt, um die schnelle, reaktive Benutzeroberfläche zu realisieren.

3.4 LiquidPlanner / Tempo Portfolio Manager

LiquidPlanner (inzwischen im Tempo-Ökosystem integriert) ist ein Enterprise-Projektmanagement-Tool. Es unterscheidet sich von den anderen Tools durch seinen Fokus auf prädiktive Planung (Predictive Scheduling) in sehr großen, komplexen IT- und Ingenieursprojekten. Statt fixer Deadlines arbeitet das System mit Wahrscheinlichkeiten (Best-Case / Worst-Case Schätzungen). Als **Enterprise-Projektmanagement-Tool** ist die Architektur auf hohe Skalierbarkeit und Sicherheit ausgelegt. Es handelt sich um ein robustes **Mandanten-Cloud-System** (SaaS), das komplexe Backend-Algorithmen für die prädiktive Planung und umfassendes Reporting nutzt.

3.5 Übersicht über themenrelevante Applikationen

Feature	TimeHero	Motion	Timeflow	TeaPot (Ziel)
Auto-Scheduling	**Ja**	**Ja (KI-basiert)**	**Nein (Visual)**	**Ja (Kernfokus)**

Zielgruppe	**Teams**	**Manager/Einzel**	**Freelancer**	**Einzelpersonen/Teams**
Plattform	**Web/Cloud**	**Web/Mobile**	**Web**	**Web (SPA)**
Besonderheit	**Kalender-Sync**	**Sofortige Re-Planung**	**UX/Drag-and-Drop**	**Ressourcenbasierte Kapazitätsplanung**

3.6 Architekturanalyse zur Technologievalidierung

Um den Vergleich zu vervollständigen und die technologische Wahl (C# Backend, React Frontend) zu untermauern, dient die folgende Architekturanalyse der themenrelevanten Applikationen als Bestätigung:

Applikation	Architektur-Muster	Plattform/Bereitstellung	Fokus der Logik	Relevanz für TeaPot
TimeHero	Single-Page-Application (SPA)	Cloud-Lösung, Browser-basiert	Echtzeit-"Auto-Adjustment" erfordert Geschwindigkeit	Bestätigt das SPA-Muster für reaktive Oberflächen (wie mit React geplant) und Cloud-Ausführung.
Motion	Client-Server-Architektur	Web- und Mobile-App, Cloud-basiert	Rechenintensive, KI-gestützte Planungslogik primär serverseitig in der Cloud	Bestätigt die Notwendigkeit einer robusten Client-Server-Struktur. Die Auslagerung der rechenintensiven Planung in das Backend unterstützt die Wahl von C# für den Algorithmus.

Timeflow	Leichtgewichtige, plattformübergreifende Anwendung	Web	Fokus auf hervorragende User Experience (UX) durch moderne Frontend-Technologien	Bestätigt, dass moderne Frontend-Frameworks (wie React) essenziell für die schnelle, reaktive Benutzeroberfläche sind.
LiquidPlanner	Robustes Mandanten-Cloud-System (SaaS)	Enterprise-Projektmanagement-Tool, Cloud-basiert	Hohe Skalierbarkeit und Sicherheit, komplexe Backend-Algorithmen	Bestätigt die Wichtigkeit eines robusten Backends (wie C#/.NET Core) zur Gewährleistung von Mandantenfähigkeit und Sicherheit.
TeaPot (Ziel)	Verteiltes Client-Server / SPA	Browserbasierte Plattform (SPA)	Algorithmische Optimierung der Aufgabenplanung und Kapazitätsauslastung	Der geplante Stack (C# Backend, React Frontend) entspricht dem modernen Industriestandard (Client-Server, SPA) und ist ideal für eine Webapplikation mit rechenintensivem Backend zur optimalen Ressourcenauslastung.

4. IT-Spezifika themenrelevanter Applikationen

Moderne Planungssysteme basieren in der Regel auf einer verteilten Client-Server-Architektur, um hohe Skalierbarkeit und ständige Erreichbarkeit zu gewährleisten.

4.1 Allgemeine technologische Komponenten

- **Frontend:** Hier dominieren Single-Page-Applications (SPAs) basierend auf Frameworks wie React, Angular oder Vue.js. Diese ermöglichen flüssige, app-ähnliche Interaktionen (wie das Verschieben von Aufgaben per Drag-and-Drop) ohne ständiges Neuladen der Seite.

- **Backend:** Die Geschäftslogik wird oft über RESTful APIs oder GraphQL bereitgestellt. Gängige Technologien sind Node.js, C# (.NET Core) oder Python. Hier liegen die rechenintensiven Planungsalgorithmen.
- **Datenbank:** Relationale Datenbanken (PostgreSQL, MySQL) kommen für die strikte Nutzer- und Rechteverwaltung zum Einsatz, oft flankiert von In-Memory-Datenbanken (wie Redis) für das schnelle Caching von Kalenderansichten.

4.2 Technologien im eigenen Projekt & Team-Kompetenz (Projekt TeaPot)

Um die Lücken in React (besonders Hooks und Context-API für das State-Management) zu schließen, wird in den ersten zwei Projektwochen eine interne Workshop-Phase durchgeführt. Parallel dazu wird die Architektur der REST-API in C# so entworfen, dass die Sicherheitsstandards (JWT-Authentifizierung) von Beginn an als Middleware integriert sind, um spätere Refactoring-Aufwände zu minimieren

4.3 Sicherheit, Datenschutz und Non-Functional Requirements

Neben den funktionalen Aspekten spielen bei einem Aufgaben-Planungssystem die nicht-funktionalen Anforderungen (Non-Functional Requirements) eine essenzielle Rolle. Da das System potenziell sensible Unternehmensdaten, persönliche Arbeitszeiten und Leistungsprofile der Mitarbeiter verarbeitet, müssen gängige Sicherheitsstandards und Vorgaben der Datenschutz-Grundverordnung (DSGVO) beachtet werden.

Auf technologischer Ebene bedeutet dies für die geplante Architektur des Projekts TeaPot:

- **Authentifizierung & Autorisierung:** Der Zugriff auf das System muss durch sichere Login-Verfahren geschützt werden. Geplant ist der Einsatz moderner Token-basierter Authentifizierung (z. B. JSON Web Tokens - JWT), um sicherzustellen, dass Benutzer ausschließlich auf die Daten ihrer zugewiesenen Teams/Gruppenzugreifen können (strikte Datentrennung pro Team).
- **Datenübertragung:** Sämtliche Kommunikation zwischen dem React-Frontend und dem C#-Backend muss zwingend verschlüsselt über HTTPS erfolgen, um das Abfangen von Aufgabendetails oder Zugangsdaten zu verhindern.
- **Datenhaltung:** Passwörter dürfen nicht im Klartext in der Datenbank gespeichert werden, sondern müssen über sichere Hashing-Verfahren (z. B. BCrypt) unkenntlich gemacht werden.

5. Analyse einer konkreten Applikation: Motion

Für die detaillierte Analyse und als Inspiration für das Projekt TeaPot wurde die Applikation "Motion" ausgewählt, da deren Kernphilosophie (automatisiertes Scheduling auf Basis von Prioritäten und Deadlines) am ehesten unserer Projektvision entspricht.

5.1 Leistungsmerkmale

Motion hebt sich durch den hohen Grad an Automatisierung von klassischen Systemen ab. Die zentralen Leistungsmerkmale sind:

- **KI-gestütztes Auto-Scheduling:** Aufgaben werden vollautomatisch in Lücken des bestehenden Kalenders geplant.
- **Soft- und Hard-Deadlines:** Das System unterscheidet zwischen "sollte erledigt sein" und "muss zwingend erledigt sein".
- **Dynamische Neuplanung:** Wird eine Aufgabe nicht rechtzeitig abgehakt oder ein Notfall-Meeting im Kalender eingetragen, berechnet der Algorithmus den gesamten Zeitplan des Nutzers im Hintergrund sofort neu.
- **Kapazitätswarnungen:** Das System blockiert die Annahme neuer Aufgaben oder schlägt Alarm, wenn das vorhandene Zeitbudget eines Mitarbeiters für die anstehenden Deadlines nicht mehr ausreicht.

5.2 Use-Cases (Anwendungsfälle)

Um die Funktionalität von Motion greifbarer zu machen, werden hier drei zentrale Use-Cases detailliert beschrieben, die so ähnlich auch für TeaPot geplant sind:

Use Case 1: Aufgabe erstellen und automatisch planen lassen

- **Akteur:** Authentifizierter Benutzer
- **Auslöser:** Der Benutzer möchte eine neue Aufgabe anlegen, die in seinen Terminkalender integriert werden soll.
- **Standardablauf:** Der Benutzer öffnet den Dialog zur Aufgabenerstellung und gibt Titel, geschätzten Zeitaufwand (z. B. 2 Stunden), Priorität und Deadline ein. Er bestätigt die Eingabe. Das System ruft im Hintergrund das Arbeitsprofil des Nutzers (Arbeitszeiten) ab und liest bereits belegte Zeitslots aus dem Kalender. Der Planungsalgorithmus identifiziert das nächste freie Zeitfenster vor Ablauf der Deadline, das groß genug für den Zeitaufwand ist.[1]
- **Systemreaktion/Ergebnis:** Die Aufgabe wird in der Datenbank gespeichert und dem Benutzer visuell als fester Block im Kalender angezeigt.[1]
- **Nachbedingung:** Die Datenbank ist konsistent aktualisiert, und der neue Termin ist für andere Teammitglieder (je nach Berechtigung) als 'belegt' sichtbar.

Use Case 2: Dynamische Umplanung bei Verzögerung

- **Akteur:** System (Motion Algorithmus) / Benutzer
- **Auslöser:** Ein geplanter Zeitslot für eine Aufgabe ist verstrichen, ohne dass der Benutzer die Aufgabe als "Erledigt" markiert hat.
- **Standardablauf:** Das System erkennt die Überschreitung des geplanten Bearbeitungszeitraums. Der Algorithmus friert Aufgaben mit höherer Priorität und

knappen Deadlines ein, sucht für die liegengebliebene Aufgabe den nächstmöglichen freien Zeitslot in der Zukunft und verschiebt dafür potenziell andere, weniger wichtige Aufgaben nach hinten.

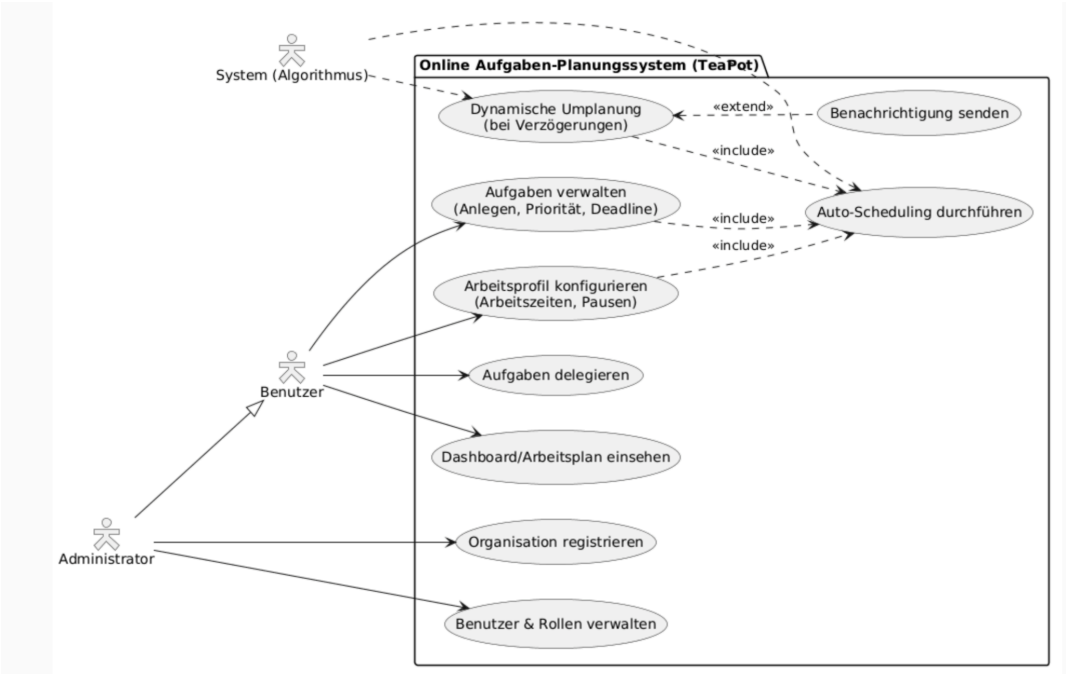
- **Systemreaktion/Ergebnis:** Der Kalender des Nutzers wird aktualisiert. Der Nutzer erhält optional eine Benachrichtigung über die Verschiebung seines Tagesplans.

Use Case 3: Arbeitsprofil und Verfügbarkeiten konfigurieren

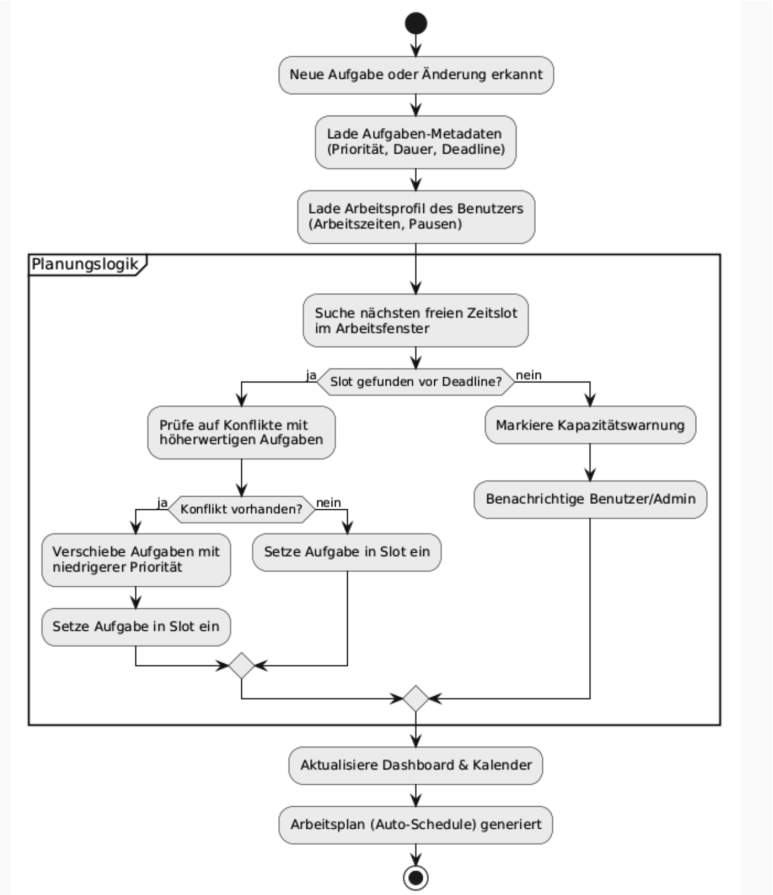
- **Akteur:** Authentifizierter Benutzer
- **Auslöser:** Ein neuer Benutzer loggt sich zum ersten Mal in das System ein oder möchte seine regulären Arbeitszeiten anpassen.
- **Standardablauf:** Der Benutzer navigiert in den Kontoeinstellungen zum Bereich "Arbeitsprofil". Er hinterlegt seine regulären Arbeitszeiten (z. B. Montag bis Freitag, 09:00 bis 17:00 Uhr) und definiert feste Blöcke für die Mittagspause (z. B. 12:30 bis 13:15 Uhr). Optional kann er eine Regel aktivieren, die besagt, dass nach 90 Minuten ununterbrochener Fokus-Arbeit automatisch ein 15-minütiger Puffer für Pausen eingeplant werden soll. Der Benutzer speichert diese Angaben.
- **Systemreaktion/Ergebnis:** Das Backend aktualisiert das Benutzerprofil in der Datenbank. Das System stößt im Hintergrund sofort einen Neuplanungsprozess (Re-Scheduling) an, um zu prüfen, ob die bereits zugewiesenen Aufgaben noch in das neu definierte Zeitraster passen. Falls Konflikte entstehen, wird der Arbeitsplan automatisch an die neuen Verfügbarkeiten angepasst und der Kalender des Nutzers visuell aktualisiert.

6. UML-Diagramme

Use-Case-Diagramm: Zeigt die Übersicht der Akteure (Benutzer, Administrator, System) und deren Interaktionsmöglichkeiten.



Aktivitätsdiagramm: Visualisiert die innere Logik des Planungsalgorithmus



7. Fazit

Die Literatur- und Marktrecherche zeigt deutlich, dass intelligente Aufgaben-Planungssysteme ein unverzichtbares Werkzeug für die moderne, digitale Arbeitsorganisation darstellen. Die Analyse von Konkurrenzprodukten wie Motion, TimeHero und Timeflow belegt, dass reine To-Do-Listen zunehmend von Systemen abgelöst werden, die den Faktor "Zeit" automatisiert managen.

Für das Projekt TeaPot lässt sich festhalten, dass die grundlegende Projektidee hochgradig relevant und praxisnah ist. Die Herausforderung für das Projektteam wird nicht in der reinen Datenspeicherung von Aufgaben liegen, sondern in der Entwicklung eines performanten und logisch korrekten Algorithmus, der Prioritäten, Deadlines und Arbeitsprofile zu einem konfliktfreien Arbeitsplan zusammensetzt. Mit dem gewählten Technologie-Stack (C# und React) ist das Team gut aufgestellt, um eine moderne, auf Ressourcenauslastungfokussierte Webapplikation zu entwerfen, wobei insbesondere im Bereich React noch Einarbeitungsaufwand eingeplant werden muss. Die Ergebnisse dieser

Recherche bilden somit eine solide, fachliche Basis für die nun folgende Spezifikationsphase und die Erstellung des Glossars und Lastenhefts.

Literatur- und Quellenverzeichnis

- **Balzert, H.; Liggesmeyer, P. (2011):** Lehrbuch der Softwaretechnik: Entwurf, Implementierung, Installation und Betrieb. 3. Aufl. Heidelberg: Spektrum Akademischer Verlag.
- **Motion (2026):** Motion – The AI Powered SuperApp for Work. Online verfügbar unter: https://www.usemotion.com/ (Zugriff: Februar 2026)
- **TimeHero (2026):** Smart Task Planning and Work Management. Online verfügbar unter: https://www.timehero.com/ (Zugriff: Februar 2026).
- **Timeflow (2026):** Smart Calendar and Task Manager. Online verfügbar unter: https://www.timeflow.site/ (Zugriff: Februar 2026).
- **Tempo (2026):** LiquidPlanner / Portfolio Manager Documentation. Online verfügbar unter: https://help.tempo.io/portfoliomanager (Zugriff: Februar 2026).
- **Microsoft (2026):** ASP.NET Core Documentation: Build web apps and services with .NET. Online verfügbar unter: https://docs.microsoft.com/en-us/aspnet/core/ (Zugriff: Februar 2026).